

Table of Contents

✓ Resolução — Exame Época Normal PAW 2024/2025	1
Parte 1 — Escolha Múltipla (8 valores).....	1
Parte 2 — Verdadeiro ou Falso (2 valores).....	8
Parte 3 — Resposta Aberta (10 valores)	9

✓ Resolução — Exame Época Normal PAW 2024/2025

Curso: LEI / LSIRC | **UC:** Programação em Ambiente Web

Data: 17/06/2024 | **Duração:** 1h30min

Nota: Não é autorizada consulta a qualquer tipo de documento

Parte 1 — Escolha Múltipla (8 valores)

Cada pergunta vale 1 valor. Indicar **todas** as opções corretas. Opção incorreta resulta em penalização de 0.5 valores.

Pergunta 1

Considere o uso da ferramenta NPM nos projetos web desenvolvidos durante o semestre. Indique todas as afirmações que indicam funcionalidades desta ferramenta:

- a. Instalar e configurar uma base de dados MongoDB;
- b. Iniciar projetos web com o comando npm start;
- c. Permite gerir dependências de projetos web;
- d. Permite criar componentes em aplicações escritas com a framework Angular.

✓ Respostas corretas: **b, c**

Justificação:

- **a) FALSO** O NPM instala *módulos/packages* JavaScript (como o mongoose), mas **não** instala nem configura a base de dados MongoDB em si. MongoDB é instalado separadamente.

- **b) VERDADEIRO** npm start executa o script “start” definido no package.json, iniciando o projeto.
 - **c) VERDADEIRO** O NPM é um **gestor de packages** para JavaScript. Permite instalar, atualizar e gerir dependências via npm install.
 - **d) FALSO** Criar componentes Angular faz-se com o CLI do Angular: ng generate component nome. O NPM apenas gere packages.
-

Pergunta 2

Indique quais dos seguintes comandos podem ser executados numa aplicação da framework Angular:

- a. ng generate component my-page;
- b. npm start;
- c. npm install mongoose --save;
- d. npm new my-app

 *Respostas corretas: a, b, c*

Justificação:

- **a) VERDADEIRO** ng generate component my-page (ou ng g component my-page) é o comando Angular CLI para criar um novo componente.
 - **b) VERDADEIRO** npm start pode ser executado em qualquer projeto Node.js/Angular para iniciar a aplicação.
 - **c) VERDADEIRO** npm install mongoose --save instala o módulo mongoose num projeto Angular (embora mongoose seja tipicamente usado no backend, o comando em si pode ser executado).
 - d) FALSO** O comando correto é ng new my-app (usa o CLI do Angular ng, não npm).
-

Pergunta 3

Indique as afirmações verdadeiras sobre os componentes do tipo serviço em Angular:

- - a. Permitem ser injetados em múltiplos componentes Angular;

- b. Podemos manter o estado global da aplicação em variáveis internas de um serviço e partilhá-las com outros vários componentes;
- c. Este componente tem de ter sempre uma interface escrita em HTML;
- d. Podem ser utilizados em conjunto com o padrão de software *observable* para manter todos os componentes visuais de uma página atualizados com o último valor das variáveis guardadas no serviço.

✓ Respostas corretas: **a, b, d**

Justificação:

- **a) VERDADEIRO** Serviços usam o decorador `@Injectable()` e são injetados via **Dependency Injection (DI)** em múltiplos componentes.
- **b) VERDADEIRO** Serviços são **singletons** (uma instância partilhada) e podem guardar estado global. Exemplo: usar `BehaviorSubject` para partilhar dados.
- **c) FALSO** Serviços **não** têm template HTML. Apenas componentes têm ficheiros `.html`. Serviços são classes TypeScript puras.
- **d) VERDADEIRO** Usando **Observables** (RxJS) e `BehaviorSubject`, os serviços propagam mudanças automaticamente via padrão `publish/subscribe`.

Pergunta 4

Considerando a framework ExpressJS utilizada em Node.js, indique as afirmações verdadeiras:

- - a. Podemos criar componentes com a linguagem de programação JavaScript e padrão de software MVC;
- - b. Podemos utilizar *template engines* para gerar páginas HTML dinamicamente no servidor;
- - c. Podemos aceder diretamente a uma base de dados para enviar e guardar informação com o módulo mongoose;
- - d. Podemos executar funções JavaScript da framework ExpressJS diretamente a partir de um browser de internet.

✓ Respostas corretas: **a, b, c**

Justificação:

- **a) VERDADEIRO** Express permite organizar aplicações com o padrão **MVC** (Models, Views, Controllers) usando JavaScript/Node.js.
 - **b) VERDADEIRO** Express suporta *template engines* como **EJS**, Pug, Mustache para gerar HTML dinâmico no servidor com `res.render()`.
 - **c) VERDADEIRO** Com o módulo **mongoose**, podemos ligar Express ao MongoDB para operações CRUD diretamente.
 - **d) FALSO** Express é uma framework **server-side**. O código Express executa no servidor Node.js, **não** diretamente no browser. O browser apenas faz pedidos HTTP ao servidor.
-

Pergunta 5

Indique todas as afirmações verdadeiras:

- a. Angular é uma framework de desenvolvimento *fullstack*;
- b. HTML é uma linguagem criada para formatar conteúdos em páginas web;
- c. Node.js é a linguagem de programação mais utilizada no desenvolvimento de aplicações Web;
- d. Angular permite a reutilização de componentes no desenvolvimento de aplicações web.

 Respostas corretas: **d**

Justificação:

- **a) FALSO** Angular é uma framework **client-side** (frontend). Para fullstack é preciso combinar com backend (Express + MongoDB = MEAN Stack).
 - **b) FALSO** HTML é uma linguagem de **marcação** (markup), não de formatação. A formatação é feita com **CSS**. HTML descreve a **estrutura** do conteúdo.
 - **c) FALSO** Node.js **não** é uma linguagem de programação — é um **runtime** (ambiente de execução) para JavaScript. A linguagem é JavaScript.
 - **d) VERDADEIRO** Um dos pontos fortes de Angular é a **reutilização de componentes**. Podemos passar dados entre componentes com `@Input()` e reutilizá-los em várias páginas.
-

Pergunta 6

Considere o excerto do ficheiro de uma página HTML presente na figura 1. Considere que este ficheiro foi aberto num browser e indique as afirmações verdadeiras:

```
<!DOCTYPE html>
<html>
<body>
  <p id="mvp">Change Me!</p>
  <p id="mvp">Change Me Again!</p>
  <button id="mvp">Executa</button>
  <script>
    function executa() {
      var elem = document.getElementById('mvp');
      elem.style.color = 'red';
    }
    document.getElementsByTagName('button')[0].addEventListener('click', executa);
  </script>
</body>
</html>
```

- a. Ao clicar no botão Executa a página não é alterada;
- b. Ao clicar no botão Executa o texto “Change Me!” altera a sua cor para vermelho;
- c. A função document.getElementById("mvp") retorna todos os elementos da página com o id é igual ao valor “mvp”;
- d. Ao clicar no botão Executa todo o texto da página fica com a cor vermelho.

✓ Respostas corretas: **b**

Justificação:

- **a) FALSO** A página é alterada: ao clicar, a função executa() é chamada e muda a cor de um elemento.
- **b) VERDADEIRO** document.getElementById('mvp') retorna o **primeiro** elemento com id="mvp", que é o <p>Change Me!</p>. A sua cor é alterada para vermelho.
- **c) FALSO** getElementById retorna **apenas um** elemento (o primeiro encontrado). Para obter múltiplos seria preciso getElementsByClassName ou querySelectorAll. Nota: ter múltiplos elementos com o mesmo id é **inválido** em HTML (IDs devem ser únicos).

- **d) FALSO** Apenas o **primeiro** <p> (“Change Me!”) fica vermelho, não todo o texto.

Pergunta 7

Considere a figura 2. Quais das seguintes afirmações são verdadeiras à luz dos exemplos estudados nas atividades letivas?

```
const verify = function (req, res, next) {
  try {
    var token = req.headers['x-access-token'];
    if (!token)
      return res.status(403).send({ auth: false, message: 'No token provided.' });
    const decoded = jwt.verify(token, config.secret)
    if (err || decoded.role !== 'ADMIN')
      return
    req.userId = decoded.id;
    next();
  } catch (exception) {
    console.log('Erro ao verificar token de autenticação');
    res.status(500).send({ auth: false, message: 'Failed to authenticate token' });
  }
}
```

- a. Está a ser usada a framework angular
- b. A função usa o módulo JWT para verificação de um token para efeitos de autenticação e autorização;
- c. A função verify é a última a ser executada numa rota da aplicação expressJS e retorna para o browser o id do utilizador ou um erro 500;
- d. Por estarmos a utilizar autenticação com tokens, estamos obrigatoriamente a utilizar uma API REST.

✓ Respostas corretas: **b**

Justificação:

- **a) FALSO** Este código usa req, res, next que são parâmetros de **middleware Express** (backend Node.js), **não** Angular.
- **b) VERDADEIRO** A função usa jwt.verify(token, config.secret) para verificar o token JWT. Faz **autenticação** (verificar token válido) E **autorização** (verificar decoded.role !== 'ADMIN').
- **c) FALSO** A função **não** é a última — é um **middleware** (tem parâmetro next()). Quando autenticação/autorização passam, chama

next() para passar ao próximo handler. Não retorna o id ao browser, guarda-o em req.userId.

- **d) FALSO** JWT pode ser usado com qualquer tipo de aplicação web, não apenas REST APIs. Pode ser usado com aplicações MPA tradicionais, WebSockets, etc.

Pergunta 8

Observe a figura 3 e indique todas as afirmações verdadeiras:

```
@Injectable({
  providedIn: 'root'
})
export class ItemRestServiceService {
  constructor(private http: HttpClient) { }

  getItem(id:string): Observable<Item> {
    return this.http.get<Item>(endpoint+'show/'+id);
  }

  addItem (item:Item): Observable<Item> {
    console.log(item);
    return this.http.post<Item>(endpoint + 'create', JSON.stringify(item),
      httpOptions);
  }
}
```

- a. Está demonstrado o consumo de uma API REST por uma aplicação escrita em ExpressJS;
- b. Está demonstrado o consumo de uma API REST por uma aplicação escrita em Angular;
- c. A função getItem retorna imediatamente com o valor do Item guardado no servidor bloqueando até estar completa;
- d. O código permite aferir com certeza absoluta que está a ser utilizada uma base de dados no backend;

✓ Respostas corretas: **b**

Justificação: - **a) FALSO** — O código usa @Injectable, HttpClient, Observable e TypeScript — tudo específico de **Angular**, não de Express. - **b) VERDADEIRO** — É um **serviço Angular** que consome uma API REST via HttpClient com métodos get e post. Usa Observable (RxJS) para programação reativa.

- **c) FALSO** A função retorna um Observable<Item>, que é **assíncrono** (não bloqueante). O valor só é obtido quando alguém faz .subscribe() no Observable.
 - **d) FALSO** O código mostra pedidos HTTP a endpoints (show/, create). Não se pode afirmar que existe uma base de dados — o backend pode guardar dados em ficheiros, memória, ou qualquer outro mecanismo.
-

Parte 2 — Verdadeiro ou Falso (2 valores)

Cada questão vale 0.5 valores. Justificar as afirmações falsas.

1. “A validação de dados de input deve ser apenas realizada no frontend de uma aplicação web para garantir que só dados corretos chegam ao backend.”

X FALSO

Justificação: A validação de dados **deve ser feita em ambos os lados** — frontend e backend.

- **Frontend:** validação para melhor UX (feedback imediato ao utilizador, ex: campos obrigatórios, formato de email)
- **Backend:** validação **obrigatória** por razões de segurança, pois o utilizador pode contornar a validação do frontend (desativando JavaScript, usando ferramentas como Postman, manipulando pedidos HTTP diretamente)

Confiar apenas na validação do frontend é uma **falha de segurança grave**.

2. “CSS é uma considerada uma linguagem de programação pois permite formatar e realizar animações em páginas HTML.”

X FALSO

Justificação: CSS (**Cascading Style Sheets**) é uma linguagem de **estilos**, não de programação. CSS não possui: - Variáveis com lógica de controlo geral (if/else, loops com lógica arbitrária) - Funções definidas pelo programador - Manipulação de dados

CSS descreve **como** os elementos HTML devem ser **apresentados** (cores, tamanhos, posicionamento, animações). Embora suporte animações com @keyframes e transições, isso não a torna uma linguagem de programação. É uma linguagem **declarativa de estilo**.

3. “O módulo JWT não é adequado para garantir autenticação e autorização em páginas web codificadas com a framework ExpressJS e o template engine EJS.”

X FALSO

Justificação: O módulo JWT (**jsonwebtoken**) é adequado e pode ser usado com ExpressJS e EJS. JWT funciona como middleware no Express para:

- **Autenticação:** verificar a identidade do utilizador através de tokens
- **Autorização:** verificar permissões (roles) do utilizador

JWT não depende do template engine utilizado. Funciona com EJS, Pug, ou qualquer outro. O token é enviado nos headers HTTP e verificado no servidor, independentemente de como as páginas são renderizadas. Na matéria estudada, JWT foi usado precisamente com Express.

4. “É possível utilizar a base de dados MongoDB no frontoffice de uma aplicação web para guardar dados da aplicação de forma persistente.”

X FALSO

Justificação: MongoDB é uma base de dados que executa no **servidor** (backend), não no frontend/frontoffice. Não é possível nem seguro ligar diretamente o frontend ao MongoDB porque: - O código do frontend fica **exposto** ao cliente

- As **credenciais** de acesso à BD ficariam visíveis - Representa um **risco de segurança** enorme

Para guardar dados persistentes no frontend, pode-se usar localStorage ou sessionStorage (Web Storage API), mas estes são mecanismos limitados do browser, não bases de dados. Para persistência real, o frontend deve comunicar com o backend (ex: via API REST) que por sua vez acede ao MongoDB.

Parte 3 — Resposta Aberta (10 valores)

Pergunta 1 (1 valor)

Indique o que entende pelo conceito cliente-servidor em aplicações web.

Resposta:

O modelo **cliente-servidor** é uma arquitetura de software onde existem dois intervenientes:

- **Cliente:** é a aplicação que faz pedidos a um servidor. No contexto web, tipicamente é o **browser** que executa código HTML, CSS e JavaScript. O cliente é responsável pela interface com o utilizador (frontend) e inicia a comunicação enviando pedidos HTTP.
- **Servidor:** é a aplicação que recebe e processa os pedidos do cliente, executando lógica de negócio, acedendo a bases de dados, e devolvendo respostas (HTML, JSON, ficheiros, etc.). No contexto da disciplina, usamos **Node.js com Express** como servidor.

O **fluxo típico** é:

1. O cliente envia um **pedido HTTP** (GET, POST, PUT, DELETE) ao servidor
2. O servidor **processa** o pedido (middleware, controllers, acesso a BD)
3. O servidor envia uma **resposta HTTP** ao cliente (HTML, JSON, código de status)
4. O cliente **renderiza** a resposta para o utilizador

Este modelo permite a **separação de responsabilidades**: o frontend lida com a apresentação e interação, enquanto o backend lida com dados, segurança e lógica de negócio. Na MEAN Stack, o Angular funciona como cliente e o Node.js/Express como servidor.

Pergunta 2 (2 valores)

Explique o que são serviços REST no âmbito do desenvolvimento de aplicações para a web.

Resposta:

REST (Representational State Transfer) é um padrão arquitetural definido por **Roy Fielding** em 2000, que define um conjunto de restrições e propriedades para a criação de **web services** baseados no protocolo HTTP.

Uma **API REST** é um conjunto de endpoints (URLs) que permitem a comunicação entre sistemas através de métodos HTTP padronizados. As operações CRUD são mapeadas para os métodos HTTP:

Método HTTP	Operação CRUD	Exemplo
GET	Read (ler)	GET /products — lista produtos
POST	Create (criar)	POST /products — cria produto
PUT	Update (atualizar)	PUT /product/:id — atualiza produto
DELETE	Delete (eliminar)	DELETE /product/:id — remove produto

Propriedades fundamentais de serviços REST:

1. **Arquitetura cliente-servidor** — separação entre cliente e servidor
2. **Stateless** cada pedido contém toda a informação necessária; o servidor não guarda estado entre pedidos
3. **Cacheable** respostas podem ser guardadas em cache para melhor desempenho
4. **Interface uniforme** URLs representam recursos e os métodos HTTP definem as operações
5. **Sistema em camadas** o cliente não precisa saber se comunica diretamente com o servidor final

Vantagens: desempenho rápido, confiabilidade, reutilização de componentes, interoperabilidade entre sistemas.

As respostas são tipicamente em formato **JSON**, facilitando o consumo por aplicações frontend (Angular, React, etc.) e outros sistemas.

No contexto prático da disciplina, implementámos APIs REST usando **Express.js** com routers separados por recurso, controllers com lógica de negócio, e modelos Mongoose para acesso ao MongoDB. Para testes, utilizámos o **Postman** e para documentação o **Swagger/OpenAPI**. Para garantir segurança, usámos **CORS** para controlar acessos cross-origin e **JWT** para autenticação via tokens.

Pergunta 3.1 (1 valor)

Identifique e corrija os erros presentes na figura 4 de forma que a página funcione corretamente.

```
<html>
<head>
  <meta charset="UTF-8">
  <title>Click Counter</title>
</head>
<body>
  <h1>Click Counter</h1>
  <p>Click Number: <span id="contador">0</span></p>
  <button>Click Me</button>
  <script>
    function contarClique() {
      numeroCliques++;
      document.getElementById(counter).textContent = numeroCliques;
    }
  </script>
</body>
</html>
```

Erros identificados e correções:

Erro 1: A variável numeroCliques nunca é **declarada nem inicializada**. -

Correção: Adicionar var numeroCliques = 0; antes da função.

Erro 2: document.getElementById(counter) — counter é usado como variável, mas deveria ser a **string** "contador" (o id do span). - **Correção:** Mudar para document.getElementById("contador").

Erro 3: O botão não tem **nenhum evento associado** — a função contarClique() nunca é chamada. - **Correção:** Adicionar onclick="contarClique()" ao botão, ou usar addEventListener.

Código corrigido:

```
<html>
<head>
  <meta charset="UTF-8">
  <title>Click Counter</title>
</head>
<body>
  <h1>Click Counter</h1>
  <p>Click Number: <span id="contador">0</span></p>
  <button onclick="contarClique()">Click Me</button>
  <script>
    var numeroCliques = 0;
    function contarClique() {
      numeroCliques++;
      document.getElementById("contador").textContent = numeroCliques;
    }
  </script>
</body>
</html>
```

Pergunta 3.2 (1 valor)

Como podemos melhorar a página para que sempre que abirmos novamente a página no mesmo browser a contagem de cliques continue e não seja reiniciada? Indique uma solução baseada apenas na página HTML apresentada.

Resposta:

Podemos usar a **Web Storage API**, especificamente o **localStorage**, para guardar o valor do contador de forma persistente no browser. O localStorage mantém os dados mesmo após fechar e reabrir o browser.

Solução:

```
<script>
  // Ao carregar a página, recuperar o valor guardado (ou 0 se não existir)
```

```
var numeroClique = parseInt(localStorage.getItem("contadorClique")) || 0;
document.getElementById("contador").textContent = numeroClique;

function contarClique() {
  numeroClique++;
  document.getElementById("contador").textContent = numeroClique;
  // Guardar o novo valor no localStorage
  localStorage.setItem("contadorClique", numeroClique);
}
</script>
```

Explicação:

- localStorage.getItem("contadorClique") — recupera o valor guardado anteriormente
 - parseInt(...) || 0 — converte para número; se não existir, usa 0
 - localStorage.setItem("contadorClique", numeroClique) — guarda o novo valor após cada clique
 - Como localStorage **persiste** entre sessões do browser (ao contrário de sessionStorage), o contador mantém-se mesmo após fechar e reabrir a página
-

Pergunta 4 (1.5 valores)

Comente a seguinte frase: “Os componentes criados na framework Angular cumprem o padrão MVC”. Na sua resposta inclua também o que sabe sobre o padrão MVC.

Resposta:

O padrão **MVC (Model-View-Controller)** é um padrão de design de software que separa uma aplicação em três componentes lógicos interligados:

- **Model (Modelo)** — Representa os dados da aplicação e a lógica de negócio. Define a estrutura dos dados (ex: classes, interfaces) e como são acedidos/manipulados.
- **View (Vista)** — Responsável pela apresentação visual dos dados ao utilizador. Define a interface (UI) e como os dados do modelo são exibidos.
- **Controller (Controlador)** — Atua como intermediário entre o Model e a View. Recebe inputs do utilizador, processa-os (interagindo com o modelo) e atualiza a vista correspondente.

A afirmação é **verdadeira**. Os componentes Angular cumprem efetivamente o padrão MVC:

Camada MVC	Componente Angular	Ficheiro
Model	Classes/Interfaces TypeScript e Serviços (@Injectable)	*.model.ts, *.service.ts
View	Template HTML do componente	*.component.html + *.component.css
Controller	Classe TypeScript do componente (@Component)	*.component.ts

Como funciona na prática:

- O ficheiro component.ts (Controller) contém a lógica, processa eventos do utilizador e comunica com serviços
- O ficheiro component.html (View) apresenta os dados usando **data binding** ({{ }}, [prop], (event), [(ngModel)])
- Os **serviços** (Model) gerem os dados, comunicam com APIs REST via HttpClient, e mantêm o estado da aplicação
- O Angular suporta **data binding bidirecional**, o que significa que alterações na View são automaticamente refletidas no Controller e vice-versa

Adicionalmente, o Angular **obriga** a esta separação pela sua própria estrutura: cada componente criado com ng generate component gera automaticamente ficheiros separados para a lógica (.ts), a vista (.html) e o estilo (.css), promovendo boas práticas de organização de código.

Pergunta 5 (1.5 valores)

Quais as funções dos componentes Guard e Intercept numa aplicação escrita com a framework Angular?

Resposta:

Guards (Guardas de Rota)

Os **Guards** são serviços Angular que implementam a interface **CanActivate** e controlam o **acesso a rotas** da aplicação. A sua principal função é verificar se um utilizador tem permissão para aceder a uma determinada página antes de a carregar.

Funcionamento:

- São declarados na configuração de routing com o parâmetro canActivate
- Antes de navegar para uma rota protegida, o Angular executa o Guard
- Se o Guard retornar true, a navegação prossegue

- Se retornar false, a navegação é bloqueada (tipicamente redirecionando para a página de login)

Exemplo prático:

```
@Injectable()
export class AuthGuard implements CanActivate {
  canActivate(): boolean {
    if (localStorage.getItem('token')) {
      return true; // Utilizador autenticado, permite acesso
    }
    this.router.navigate(['/login']); // Redireciona para login
    return false;
  }
}
```

// No routing:

```
{ path: 'admin', component: AdminComponent, canActivate: [AuthGuard] }
```

Caso de uso: Proteger páginas que requerem autenticação (ex: painel de administração, páginas de edição).

Interceptors (Interceptadores)

Os **Interceptors** são serviços Angular que implementam a interface **HttpInterceptor** e permitem **interceptar e modificar pedidos HTTP** antes de serem enviados ao servidor, e/ou modificar respostas antes de chegarem aos componentes.

Funcionamento:

- São declarados no app.module.ts na secção providers
- Todos os pedidos HTTP feitos pelo HttpClient passam automaticamente pelos interceptors
- Implementam o método intercept(req, next) que pode modificar o pedido e chamar next.handle() para o encaminhar

Exemplo prático:

```
@Injectable()
export class AuthInterceptor implements HttpInterceptor {
  intercept(req: HttpRequest<any>, next: HttpHandler): Observable<HttpEvent<any>> {
    const token = localStorage.getItem('token');
    const clonedReq = req.clone({
      headers: req.headers.set('Authorization', 'Bearer ' + token)
    });
    return next.handle(clonedReq);
  }
}
```

```
}  
}
```

Caso de uso: Adicionar **automaticamente** o token JWT a **todos** os pedidos HTTP, sem ter de o fazer manualmente em cada serviço. Também podem ser usados para logging, tratamento de erros globais, ou transformação de dados.

Em resumo: Os Guards protegem o acesso a **rotas/páginas** (autorização de navegação), enquanto os Interceptors modificam **pedidos HTTP** (ex: adicionar headers de autenticação). Ambos são fundamentais para implementar autenticação e autorização numa aplicação Angular.